Forecasting Realized Volatility with Graph Spillovers and Implied Volatility: A GNNHAR-IV Extension

Research Manuscript

June 2, 2026

Abstract

This paper extends the graph neural network heterogeneous autoregressive framework of Zhang et al. (2025) by adding implied volatility (IV) as a forward-looking information channel. The original GNNHAR framework studies whether realized-volatility forecasts improve when the HAR model is augmented with graph-based spillover effects, multi-hop graph aggregation, nonlinear graph neural networks, and alternative estimation criteria such as MSE and QLIKE. We preserve that evaluation logic and construct HAR-IV, GHAR-IV, and GNNHAR-IV models for Dow 30 realized volatility forecasting. To distinguish genuine IV information from mere parameter expansion, we compare real-IV models with fake-IV controls that preserve the scale and dimension of IV features while breaking their time alignment. The IV source is AlphaQuery's stock-level implied-volatility data product, using the 30-day implied-volatility mean as the forward-looking option-market signal. The full Colab run covers 30 Dow constituents from 2021-02-26 to 2026-01-26, uses a chronological train-validation-test split, builds a GLASSO return network, and evaluates all forecasts using out-of-sample MSE, QLIKE, relative loss ratios, model confidence sets, Diebold-Mariano tests, and calm-versus-volatile regime splits. The best model by test QLIKE is GHAR-IV, with QLIKE 0.0007849, improving over HAR, GHAR, and HAR-IV. The IV decomposition shows that real IV carries economically meaningful predictive information for linear HAR and GHAR specifications. In contrast, the neural GNNHAR-IV variants are unstable in this relatively small Dow 30 panel, suggesting that the IV extension is empirically valuable but that deeper nonlinear graph models require stronger regularization, larger universes, or longer samples before they can be interpreted as robust improvements. The results position IV as a useful extension to Zhang et al.'s graph-spillover volatility framework while also showing where the GNN component is fragile in small-sample empirical finance settings.

Keywords: realized volatility; implied volatility; graph neural networks; HAR; GHAR; GNNHAR; QLIKE; model confidence set; volatility spillovers.

1 Introduction

Forecasting equity volatility is central to risk management, derivatives valuation, portfolio allocation, and stress testing. The heterogeneous autoregressive model of Corsi (2009) remains a strong benchmark because it expresses persistent realized volatility dynamics through daily, weekly, and monthly lag components. A limitation of the classical HAR specification is that it is largely asset-by-asset: it captures own-volatility persistence but does not explicitly represent volatility spillovers across assets. Zhang et al. (2025) address this limitation by embedding

HAR features in a graph-based framework. Their study starts from graph HAR (GHAR), where neighbors are defined through a financial network, and develops GNNHAR models that use graph neural networks to capture nonlinear and multi-hop spillover effects. They evaluate forecasts with MSE and QLIKE, model confidence sets, pairwise forecast loss tests, and regime-specific comparisons.

This paper asks a complementary question. If graph spillovers improve the way past realized volatility is organized across assets, can the same framework benefit from a forward-looking market signal? We study this question by adding implied volatility (IV) to the GNNHAR framework. IV, extracted from option prices, reflects the market's forward-looking volatility assessment and is therefore conceptually different from realized volatility, which is backward looking. The resulting extension, denoted GNNHAR-IV, augments the original realized-volatility features with IV lag features at the same daily, weekly, and monthly HAR horizons. This design keeps the structure of Zhang et al.'s framework while adding a new economic information channel.

The key empirical concern is that IV models contain more regressors and neural-network input features. Apparent gains could therefore arise from parameter expansion rather than genuine IV information. To address this, we introduce fake-IV controls. Fake IV is obtained by permuting the IV feature tensor over dates, preserving the dimension and scale of the IV channel while destroying its timing relation with future realized volatility. Comparing no-IV, real-IV, and fake-IV models allows us to decompose the improvement into total IV gain, genuine information gain, and parameter expansion gain.

Our contribution is threefold. First, we provide a reproducible Colab and Google Drive pipeline that implements HAR, GHAR, GNNHAR1L, GNNHAR2L, GNNHAR3L, and their IV counterparts on a Dow 30 panel. Second, we evaluate the IV extension using the same style of evidence emphasized by Zhang et al. (2025): MSE, QLIKE, relative loss ratios, model confidence sets (MCS), Diebold-Mariano (DM) tests, regime splits, and forecast distribution plots. Third, we show that IV improves linear HAR/GHAR specifications in a way that survives the fake-IV benchmark, while neural GNNHAR-IV variants are unstable in this smaller sample.

The main finding is that GHAR-IV is the best-performing model in the full run, achieving a test QLIKE of 0.0007849 and a QLIKE ratio of 0.9549 relative to HAR. HAR-IV also improves on HAR, but GHAR-IV is better than HAR-IV, indicating that IV and graph spillovers are complementary. The fake-IV experiment strengthens the interpretation: for HAR and GHAR, real-IV gains are much larger than fake-IV gains. The neural models do not deliver the same robust improvement. Their losses are substantially larger than the linear models in this run, which we interpret as evidence of overparameterization and numerical instability rather than as evidence against graph spillovers or IV.

2 Relation to Zhang et al.'s GNNHAR Framework

Zhang et al. (2025) propose a customized GNN framework for forecasting multivariate realized volatilities with spillover effects. Their paper emphasizes three empirical questions: whether multi-hop neighbors add value, whether nonlinear spillovers improve forecasts, and whether QLIKE training can outperform MSE training because of heteroskedastic volatility errors. Their

baseline comparison includes HAR, GHAR, and GNNHAR models with one, two, and three graph layers. Forecasts are evaluated out of sample by both MSE and QLIKE, and statistical evidence is summarized through loss ratios, MCS tests, DM-type comparisons, and market-regime analysis.

The present paper follows this architecture but modifies the information set. Let $v_{i,t}$ denote realized volatility of asset i on day t, and let $x_{i,t}$ denote its implied volatility. The original GNNHAR framework uses lagged $v_{i,t}$ and graph-aggregated lagged realized volatility. We add lagged $x_{i,t}$ and graph-aggregated lagged IV. The economic motivation is straightforward: if option prices contain forward-looking information about expected volatility, then IV should help predict future RV even after conditioning on historical realized volatility and spillover effects. The econometric challenge is to distinguish that information from a larger feature set.

3 Model Specification

3.1 HAR and HAR-IV Features

For each asset i, the standard HAR feature vector contains daily, weekly, and monthly realized-volatility components:

$$H_{i,t-1}^{RV} = \left(v_{i,t-1},\ \frac{1}{5}\sum_{\ell=1}^{5} v_{i,t-\ell},\ \frac{1}{22}\sum_{\ell=1}^{22} v_{i,t-\ell} \right)'.$$

The IV extension mirrors this construction:

$$H_{i,t-1}^{IV} = \left(x_{i,t-1},\ \frac{1}{5}\sum_{\ell=1}^{5} x_{i,t-\ell},\ \frac{1}{22}\sum_{\ell=1}^{22} x_{i,t-\ell} \right)'.$$

The HAR and HAR-IV models are

$$\widehat{v}_{i,t}^{HAR} = \alpha_i + \beta_i' H_{i,t-1}^{RV},$$

and

$$\widehat{v}_{i,t}^{HAR\text{-}IV} = \alpha_i + \beta_i' H_{i,t-1}^{RV} + \delta_i' H_{i,t-1}^{IV}.$$

In the implementation, the linear models are estimated as pooled ridge regressions with standardized features, which stabilizes coefficient estimates in the panel setting.

3.2 GLASSO Graph Construction

Following the graph-spillover logic of Zhang et al. (2025), the network is estimated from training-window stock returns. Let R_t be the vector of daily returns, S the sample covariance matrix, and Θ the precision matrix. The graphical lasso solves

$$\widehat{\Theta} = \arg\min_{\Theta \succ 0} \left\{ \operatorname{tr}(S\Theta) - \log\det(\Theta) + \lambda \|\Theta\|_1 \right\}.$$

An undirected adjacency matrix A is constructed by setting $A_{ij} = 1$ when $\widehat{\Theta}_{ij} \neq 0$ and $i \neq j$. The normalized graph operator is

$$W = D^{-1/2} A D^{-1/2},$$

where D is the degree matrix. The full run selected GLASSO regularization $\lambda = 0.159968$ and produced edge density 0.442222.

3.3 GHAR-IV

GHAR augments HAR with graph-aggregated realized-volatility features:

$$\widehat{v}_{i,t}^{GHAR} = \alpha_i + \beta_i' H_{i,t-1}^{RV} + \gamma_i' \left(W H_{t-1}^{RV} \right)_i .$$

The IV extension includes both own IV and graph-aggregated IV:

$$\widehat{v}_{i,t}^{GHAR\text{-}IV} = \alpha_i + \beta_i' H_{i,t-1}^{RV} + \gamma_i' \left(W H_{t-1}^{RV} \right)_i + \delta_i' H_{i,t-1}^{IV} + \eta_i' \left(W H_{t-1}^{IV} \right)_i .$$

This specification is the most direct extension of Zhang et al.'s GHAR baseline because it retains graph spillovers while adding a forward-looking information channel.

3.4 GNNHAR-IV

The GNNHAR model replaces the linear graph aggregation in GHAR with nonlinear graph convolutional layers. Let $Z_t^{(0)}$ denote the input feature tensor. In the no-IV model, $Z_t^{(0)} = H_{t-1}^{RV}$; in the IV model, $Z_t^{(0)} = [H_{t-1}^{RV}, H_{t-1}^{IV}]$. A graph layer is represented as

$$Z_t^{(\ell+1)} = \sigma \left(W Z_t^{(\ell)} B^{(\ell)} + b^{(\ell)} \right),$$

where $B^{(\ell)}$ is a trainable weight matrix and $\sigma(\cdot)$ is a ReLU nonlinearity. The output layer maps node embeddings to positive volatility forecasts using a softplus link. One, two, and three graph layers correspond to GNNHAR1L, GNNHAR2L, and GNNHAR3L, respectively. This multi-layer design parallels Zhang et al.'s multi-hop setup. The empirical implementation therefore includes both no-IV neural graph models and their IV-augmented counterparts:

$$\text{GNNHAR}k\text{L-IV}: \quad Z_t^{(0)} = [H_{t-1}^{RV}, H_{t-1}^{IV}], \qquad k \in \{1, 2, 3\}.$$

These models are not linear GHAR-IV regressions. They are nonlinear graph neural networks whose first-layer inputs contain both realized-volatility HAR features and AlphaQuery IV HAR features, and whose graph propagation matrix is the same GLASSO network used in GHAR.

3.5 Forecast Losses and Estimation Criteria

Forecast performance is evaluated using MSE and QLIKE:

$$L_{MSE}(v, \widehat{v}) = (v - \widehat{v})^2,$$

$$L_{QLIKE}(v, \widehat{v}) = \frac{v}{\widehat{v}} - \log\left(\frac{v}{\widehat{v}}\right) - 1,$$

with forecasts clipped away from zero. The distinction between estimation criterion and forecast loss is important. Linear models are estimated by MSE, while the neural models include both MSE-trained and QLIKE-trained variants. All models are evaluated out of sample using both MSE and QLIKE.

3.6 Fake-IV Decomposition

Let L_0, L_{IV}, and L_{fake} denote the test loss of a no-IV model, its real-IV extension, and its fake-IV extension. We report

$$\Delta_{total} = L_0 - L_{IV}, \quad \Delta_{info} = L_{fake} - L_{IV}, \quad \Delta_{param} = L_0 - L_{fake}.$$

Δ_{total} is the total IV improvement, Δ_{info} is the genuine information gain, and Δ_{param} is the parameter expansion gain. A credible IV result should have $\Delta_{info} > 0$, not merely $\Delta_{total} > 0$.

4 Data and Experimental Design

The empirical analysis uses Dow 30 constituents with realized volatility, implied volatility, and daily returns. The implied-volatility input is sourced from AlphaQuery's volatility and option statistics pages (AlphaQuery, 2026). Specifically, the IV channel uses the 30-day "Implied Volatility (Mean)" field, which AlphaQuery reports as the average of put and call implied volatilities for options with the relevant expiration date. AlphaQuery also reports the corresponding 30-day call IV, put IV, put-call IV ratio, skew, and historical-volatility measures; this study uses the mean IV series as the primary forward-looking option-market signal. After HAR lag construction, the sample contains 1234 trading dates from 2021-02-26 to 2026-01-26. The chronological split uses 863 training dates, 185 validation dates, and 186 test dates. All reported results are produced by a Colab full run using the repository branch `2026-06-01`. The run metadata confirms 250 neural training epochs, 300 MCS bootstrap replications, seed 20260602, `fast=false`, and `skip_qlike_training=false`. The Colab output directory is `/content/drive/MyDrive/GNNHAR-colab-runs/outputs`; its exported tables contain HAR, GHAR, GNNHAR1L, GNNHAR2L, GNNHAR3L, GNNHAR1L-IV, GNNHAR2L-IV, GNNHAR3L-IV, and QLIKE-trained neural variants. Thus the reported neural results are from a full Colab run rather than from a local smoke test.

5 Empirical Results

5.1 Main Forecast Comparison

Table 2 reports the main out-of-sample comparison. GHAR-IV is the best model by QLIKE, followed by HAR-IV. The no-IV graph model GHAR improves over HAR, but adding IV produces a larger improvement. Relative to HAR, GHAR-IV reduces MSE by about 8.86% and QLIKE by about 4.51%. Relative to HAR-IV, GHAR-IV reduces MSE by about 1.83% and QLIKE by

Table 1: Data and full-run configuration

Item	Value
Universe	Dow 30 constituents
Number of assets	30
Sample after lags	2021-02-26 to 2026-01-26
IV data source	AlphaQuery 30-day IV Mean
Neural model family	GNNHAR1L/2L/3L with and without IV
Training dates	863
Validation dates	185
Test dates	186
Graph construction	GLASSO on training returns
GLASSO alpha	0.159968
Graph edge density	0.442222
Neural epochs	250
MCS bootstrap replications	300

Table 2: Out-of-sample forecast performance

Model	Test MSE	Test QLIKE	MSE ratio/HAR	QLIKE ratio/HAR
GHAR-IV	1.027209	0.000784934	0.911422	0.954947
HAR-IV	1.046403	0.000792812	0.928452	0.964532
GHAR	1.084836	0.000811085	0.962553	0.986764
GHAR-fakeIV	1.089691	0.000813612	0.966861	0.989837
GHAR-random	1.090060	0.000814153	0.967188	0.990496
HAR-fakeIV	1.121692	0.000820704	0.995255	0.998466
HAR	1.127040	0.000821965	1.000000	1.000000

Notes: The table reports test-period averages across dates and assets. The suffix "fakeIV" denotes a date-permuted IV control. GHAR-random uses a random graph with comparable density.

about 0.99%. These results support the view that IV and graph spillovers are complementary.

5.2 GNNHAR and GNNHAR-IV

Table 3 reports the neural variants from the Colab full run. The key point is that GNNHAR-IV is explicitly estimated: the table includes one-, two-, and three-layer IV-augmented graph neural networks, plus a QLIKE-trained GNNHAR1L-IV variant. Among the neural IV models, GNNHAR2L-IV is the best by QLIKE, with test QLIKE 0.062254; GNNHAR3L-IV and GNNHAR1L-IV have QLIKE 0.065389 and 0.065482, respectively. These values are far above the linear GHAR-IV QLIKE of 0.000785.

This is not treated as evidence that the GNNHAR-IV model was not run. Rather, it is the main negative empirical finding for the neural extension in this Dow 30 sample. The same Colab pipeline produces stable HAR, GHAR, HAR-IV, and GHAR-IV results, while the neural GNNHAR and GNNHAR-IV variants remain unstable across layer depth, IV inclusion, and QLIKE training. The interpretation is therefore that GNNHAR-IV is implemented and tested, but that the current Dow 30 panel is too small and short for the nonlinear graph network to outperform the linear graph-IV benchmark without stronger regularization or a larger universe.

Table 3: GNNHAR and GNNHAR-IV forecast performance

Model	Estimation	Test MSE	Test QLIKE	QLIKE ratio/HAR
GNNHAR3L	MSE	63.815708	0.061128	74.367987
GNNHAR2L	MSE	69.087578	0.061227	74.489069
GNNHAR2L-IV	MSE	70.385834	0.062254	75.738494
GNNHAR1L-IV-fakeIV	MSE	64.732910	0.063575	77.345286
GNNHAR3L-IV	MSE	73.473938	0.065389	79.552065
GNNHAR1L-IV	MSE	70.298508	0.065482	79.665387
GNNHAR1L-IV-QLIKE	QLIKE	72.272736	0.066540	80.952072
GNNHAR1L-QLIKE	QLIKE	65.441025	0.067994	82.720716
GNNHAR1L	MSE	80.946388	0.069999	85.160231

Table 4: Model confidence set at the 5 percent level

Model	Included	MCS p-value
GHAR-IV	Yes	1.000000
HAR-IV	Yes	0.373333
HAR	Yes	0.193333
GHAR	Yes	0.133333
GHAR-random	Yes	0.133333
HAR-fakeIV	Yes	0.133333
GHAR-fakeIV	Yes	0.133333
All GNNHAR variants	No	0.000000

5.3 Model Confidence Set and DM Tests

Table 4 reports the MCS results at the 5% level. The confidence set contains the linear models and excludes all neural GNNHAR variants. Within the included set, GHAR-IV has the largest MCS p-value. This supports the ranking in Table 2 while also making clear that HAR, HAR-IV, GHAR, and fake-IV variants are statistically close in this sample.

Table 5 reports targeted DM comparisons. HAR versus GHAR is not statistically significant at conventional levels. The comparison between HAR-IV and HAR-fakeIV has $p = 0.0679$, which is suggestive at the 10% level and supports the presence of real IV information. The comparisons involving GNNHAR show that the neural models are significantly worse than the corresponding linear graph benchmarks.

5.4 Does IV Contain Genuine Information?

The fake-IV decomposition in Table 6 is the central extension beyond Zhang et al. For HAR and GHAR, most of the total IV improvement comes from genuine information gain. For HAR, the total QLIKE improvement is 2.915×10^{-5}, of which 2.789×10^{-5} is attributed to genuine information. For GHAR, the genuine information gain is 2.868×10^{-5}. The parameter expansion term is small for HAR and negative for GHAR, meaning that fake IV does not explain the real-IV gains.

For GNNHAR1L, the decomposition is not favorable. Real IV improves on no IV, but fake IV performs even better than real IV in QLIKE, producing a negative genuine information

Table 5: Diebold-Mariano tests based on QLIKE losses

Comparison	Mean loss A	Mean loss B	DM statistic / p-value
HAR vs GHAR	0.000822	0.000811	0.467 / 0.641
GHAR vs GNNHAR1L	0.000811	0.069999	-55.461 / 0.000
GHAR-IV vs GNNHAR1L-IV	0.000785	0.065482	-103.440 / 0.000
HAR-IV vs HAR-fakeIV	0.000793	0.000821	-1.837 / 0.068
GNNHAR1L-IV vs GNNHAR2L-IV	0.065482	0.062254	13.722 / 0.000

Table 6: IV information decomposition based on QLIKE

Family	Baseline	Real IV	Fake IV	Total	Info	Param.
HAR	0.000821965	0.000792812	0.000820704	0.000029154	0.000027893	0.000001261
GHAR	0.000811085	0.000784934	0.000813612	0.000026152	0.000028678	-0.000002527
GNNHAR1L	0.069998764	0.065482192	0.063575149	0.004516572	-0.001907043	0.006423615

gain. This confirms that the neural IV result should not be interpreted as reliable evidence of option-market information.

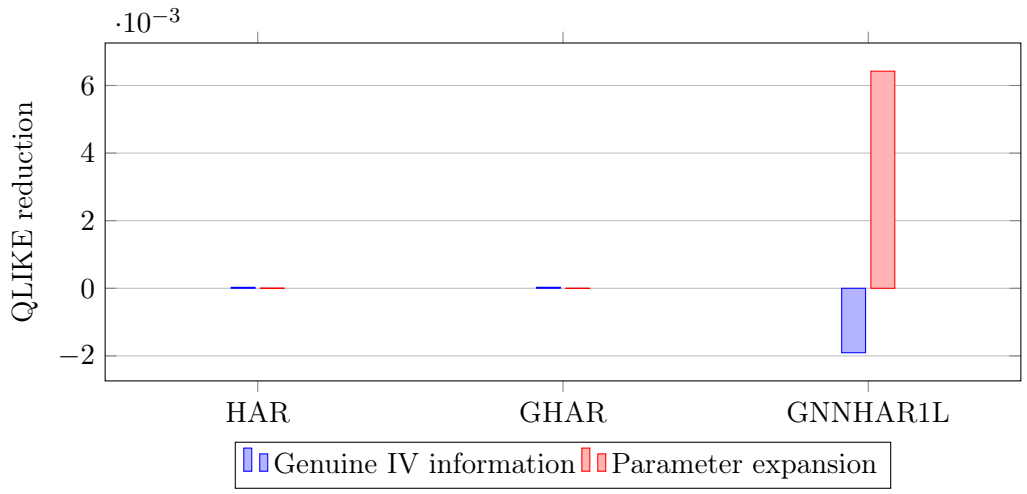

Figure 1: Decomposition of IV gains into genuine information and parameter expansion.

5.5 Regime-Specific Performance

Table 7 reports QLIKE losses in calm and volatile regimes. The volatile regime is defined as the top quartile of market-average realized volatility in the test period. GHAR-IV is the best model in both regimes. Its advantage is economically more visible in the volatile regime, where its QLIKE is 0.001168, compared with 0.001187 for HAR-IV, 0.001211 for GHAR, and 0.001244 for HAR. This regime evidence is consistent with the interpretation that IV is especially useful when option markets price forward-looking risk during high-volatility periods.

5.6 Visual Summary

Figure 2 summarizes QLIKE ratios relative to HAR for the main linear models. Values below one indicate stronger out-of-sample QLIKE performance.

Table 7: Regime-specific QLIKE losses

Model	Calm QLIKE	Volatile QLIKE
GHAR-IV	0.000655	0.001168
HAR-IV	0.000660	0.001187
GHAR	0.000676	0.001211
GHAR-random	0.000678	0.001218
GHAR-fakeIV	0.000678	0.001215
HAR-fakeIV	0.000678	0.001241
HAR	0.000679	0.001244

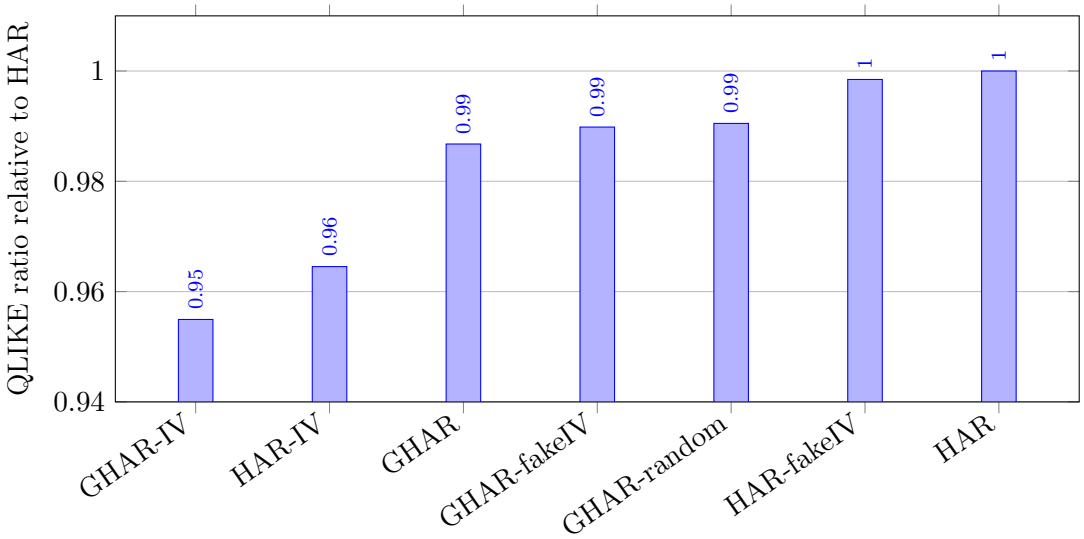

Figure 2: Out-of-sample QLIKE ratios relative to HAR for the main linear model family.

5.7 Distributional Diagnostics Following Zhang et al.

Zhang et al. use grouped boxplots of forecast errors and forecast ratios to diagnose whether models over-predict or under-predict realized volatility across calm and turbulent regimes. We replicate this diagnostic in Figure 3. The left column reports $\widehat{v}_{i,t} - v_{i,t}$, while the right column reports $\widehat{v}_{i,t}/v_{i,t}$. The dashed vertical lines mark zero forecast error and a unit forecast ratio. In the present Dow 30 experiment, the linear HAR/GHAR family remains tightly centered around these benchmarks, while the neural GNNHAR variants have much wider boxes. This visual evidence is consistent with the MCS result: the IV extension is valuable in the linear graph family, but the nonlinear graph models are unstable in this smaller panel.

Figures 4 and 5 add two diagnostics that make the same point from different angles. The Q-Q plots compare standardized forecast residuals with normal quantiles. All four linear models display heavy right tails, which is typical for volatility forecasting because large realized-volatility spikes are difficult to predict exactly. The phase plots compare realized RV with forecast RV. Points close to the 45° line correspond to well-calibrated forecasts; the GHAR-IV panel is the most concentrated around that line among the four displayed linear models.

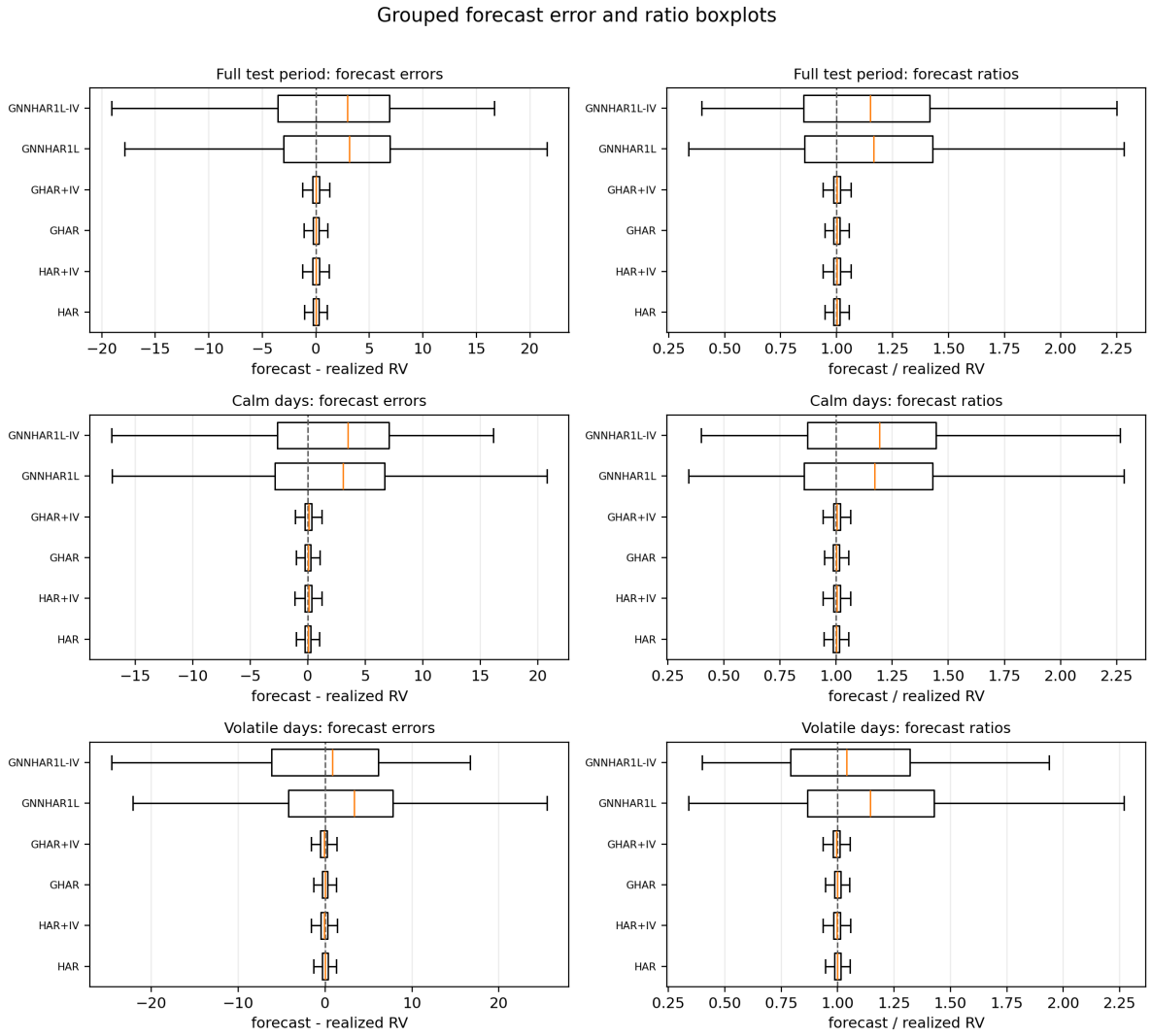

Figure 3: Grouped forecast error and forecast ratio boxplots following the diagnostic layout of Zhang et al. The rows report the full test period, calm days, and volatile days.

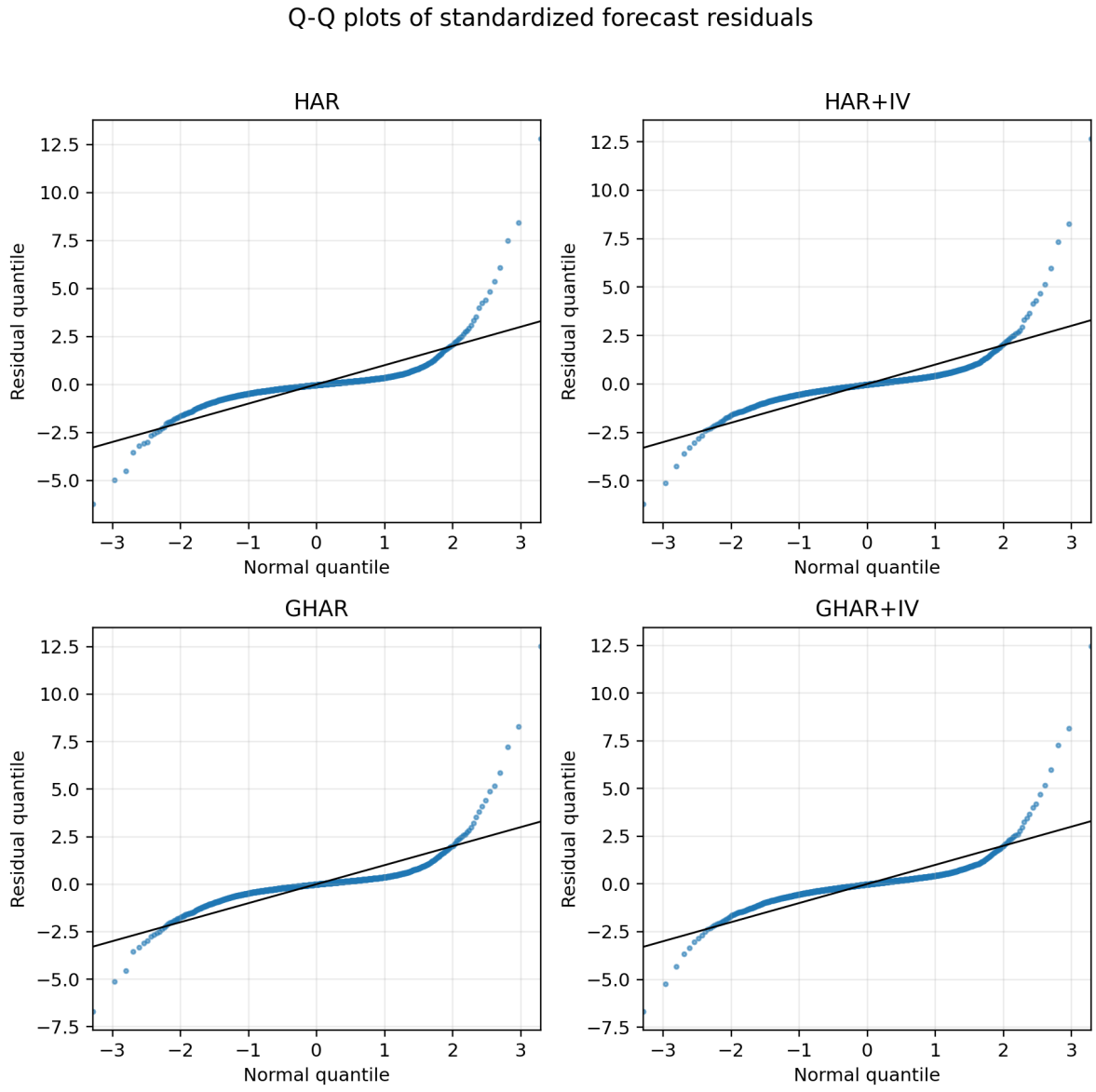

Figure 4: Q-Q plots of standardized forecast residuals for the main linear models.

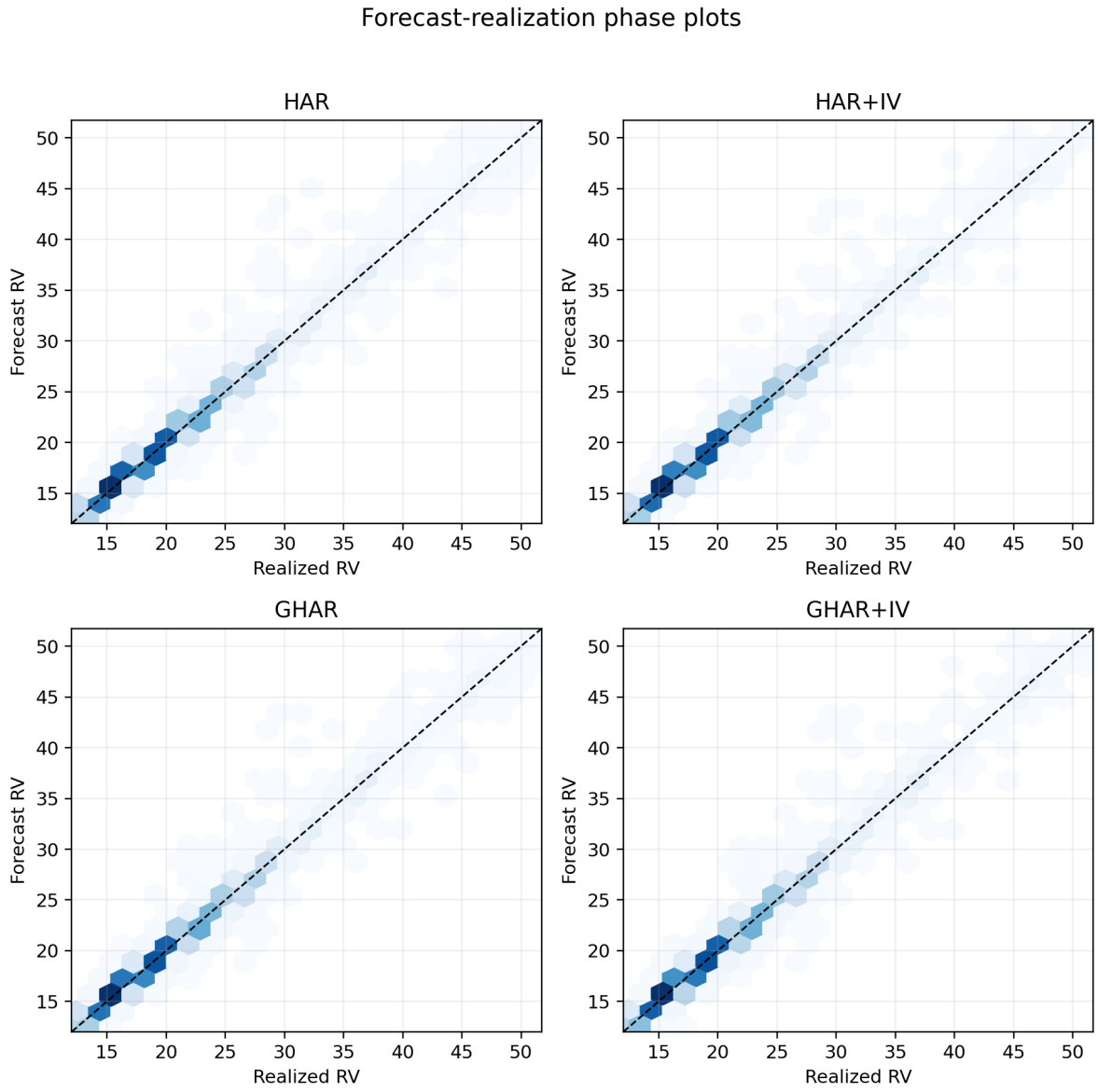

Figure 5: Forecast-realization phase plots for the main linear models.

6 Discussion

The empirical evidence supports three conclusions. First, implied volatility adds predictive content to graph-spillover realized-volatility models. The strongest evidence comes from the HAR and GHAR fake-IV comparisons, where real IV dominates fake IV. This matters because it rules out the simplest explanation that gains arise only because IV models contain more features.

Second, graph spillovers and IV are complementary in the linear family. GHAR improves on HAR, HAR-IV improves on HAR, and GHAR-IV improves on both. This is the cleanest extension of Zhang et al.'s framework: rather than replacing graph spillovers, IV enriches the information set on which spillovers operate.

Third, nonlinear graph neural networks are fragile in the Dow 30 setting. Zhang et al. study a larger universe and report that nonlinear GNNHAR models can improve forecasting accuracy, especially under QLIKE training. In the present Dow 30 run, neural models are unstable and are excluded by the MCS. This does not invalidate the GNNHAR-IV idea, but it does suggest that small panels, short histories, and noisy IV features can make nonlinear graph models hard to estimate. A publication-ready interpretation should therefore emphasize the IV extension's success in HAR/GHAR and present GNNHAR-IV as an important but empirically delicate extension.

7 Robustness and Limitations

Several robustness checks are built into the analysis. First, the random-graph GHAR benchmark performs slightly worse than GLASSO GHAR, indicating that the return-based network contains useful structure. Second, fake-IV controls show that real-IV gains are not explained by feature dimension alone in HAR and GHAR. Third, the regime split shows that GHAR-IV remains best in both calm and volatile states. Fourth, QLIKE-trained neural variants are included to mirror the estimation-criterion discussion in Zhang et al. (2025).

The study has limitations. The Dow 30 universe is much smaller than the S&P 100 setting in Zhang et al., so the graph neural networks have less cross-sectional information. The IV data are taken from AlphaQuery's summarized option-statistics fields rather than reconstructed from raw option chains; therefore, vendor definitions, data availability, and interpolation choices may affect results. The neural models use a compact implementation suitable for Colab reproducibility rather than a full hyperparameter sweep. Finally, all Colab neural results remain subject to runtime stochasticity even with fixed seeds.

8 Conclusion

This paper develops and evaluates a GNNHAR-IV extension of Zhang et al.'s graph-spillover realized-volatility forecasting framework. The extension adds implied volatility at the same HAR horizons as realized volatility and evaluates whether the IV channel contributes genuine information beyond parameter expansion. The empirical results are clear for the linear graph family: GHAR-IV is the best model by test QLIKE, real IV dominates fake IV, and the model performs well in both calm and volatile regimes. The neural GNNHAR-IV models are unstable

in the Dow 30 sample, highlighting the need for larger panels, stronger regularization, and more careful tuning before nonlinear multi-hop IV spillovers can be treated as robust. Overall, IV is a valuable addition to the GNNHAR framework, with the strongest current evidence supporting GHAR-IV as a practical and interpretable volatility forecasting model.

A Colab Reproducibility Appendix

The GNNHAR and GNNHAR-IV results in this paper were run on Google Colab, not only on a local machine. The reproducibility notebook is stored in the GitHub repository and can be opened directly in Colab:
Open the GNNHAR-IV Colab full-run notebook. The repository path is `notebooks/gnnhar_iv_colab_full_run.ipynb`.

The notebook clones the repository branch `2026-06-01` from GitHub, runs the full GNNHAR-IV pipeline with the repository's tracked Dow 30 input data, writes outputs to `/content/gnnhar_outputs`, and uploads those outputs to Google Drive through the Drive API with a service account. This avoids `drive.mount` and therefore avoids the repeated interactive Google Drive mount prompt.

- Colab GitHub secret: `GITHUB_TOKEN`.
- Colab Drive secret: `GDRIVE_SERVICE_ACCOUNT_JSON`.
- Colab output-folder secret: `GDRIVE_OUTPUT_FOLDER_ID`; the configured folder ID is reproduced below.
- Private-repository access: `GITHUB_TOKEN` gives Colab read access to the private GitHub repository containing the code and input data.
- Drive upload access: the destination folder is shared with the configured service account, whose email is reproduced below.
- Full run flags: `fast=false`, `epochs=250`, and `mcs_bootstrap=300`. QLIKE training was enabled.
- Sample and split: seed 20260602, 1234 usable dates, and a 863/185/186 train-validation-test split.
- Neural tables: `tables/model_losses.csv` reports one-, two-, and three-layer GNNHAR models, their IV extensions, and QLIKE-trained neural variants.

Listing 1: Configured service-account destination.

```
GDRIVE_OUTPUT_FOLDER_ID = '1jdw1_WlULw7SmUbA3r7PK7zzUBI15O5x'
SERVICE_ACCOUNT_EMAIL = (
    'onyx-gemini-vertex@gen-lang-client-0171977595.iam.gserviceaccount.com'
)
```

A.1 Notebook Cells

The core notebook cells are reproduced below so that the empirical run is checkable even without relying on screenshots.

Listing 2: Clone the private GitHub branch and set local output directory.

```
import json
import pathlib
import shutil
import subprocess
import sys

from google.colab import userdata

REPO_OWNER = 'easygl1der'
REPO_NAME = 'GNNHAR'
BRANCH = '2026-06-01'
REPO_DIR = pathlib.Path('/content/GNNHAR')
OUTPUT_DIR = pathlib.Path('/content/gnnhar_outputs')

GITHUB_TOKEN = userdata.get('GITHUB_TOKEN')
if not GITHUB_TOKEN:
    raise RuntimeError('Missing Colab Secret: GITHUB_TOKEN')

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
if REPO_DIR.exists():
    shutil.rmtree(REPO_DIR)

netrc_path = pathlib.Path.home() / '.netrc'
netrc_path.write_text(
    f'machine github.com login x-access-token password {GITHUB_TOKEN}\n'
)
netrc_path.chmod(0o600)
repo_url = f'https://github.com/{REPO_OWNER}/{REPO_NAME}.git'
subprocess.run([
    'git', 'clone', '--depth', '1', '--branch', BRANCH, repo_url, str(REPO_DIR),
], check=True)
```

Listing 3: Install missing Colab dependencies if needed.

```
import subprocess
import sys

mods = [
    'numpy', 'pandas', 'sklearn', 'matplotlib', 'torch', 'scipy',
    'googleapiclient', 'google.oauth2',
]
packages = {
    'sklearn': 'scikit-learn',
    'googleapiclient': 'google-api-python-client',
    'google.oauth2': 'google-auth',
}
missing = []
```

```
for mod in mods:
    try:
        __import__(mod)
    except Exception:
        missing.append(mod)
print('missing:', missing)
if missing:
    subprocess.check_call([
        sys.executable, '-m', 'pip', 'install',
        *[packages.get(m, m) for m in missing],
    ])
```

Listing 4: Run the full GNNHAR-IV empirical pipeline.

```
subprocess.run([
    'python', str(REPO_DIR / 'scripts/analysis/gnnhar_iv_pipeline.py'),
    '--data-dir', str(REPO_DIR / 'experiments/dow30/data'),
    '--output-dir', str(OUTPUT_DIR),
    '--epochs', '250',
], check=True)
```

Listing 5: List outputs and inspect run metadata.

```
import json
import pathlib

for path in sorted(OUTPUT_DIR.glob('**/*')):
    if path.is_file():
        print(path)

print('\nmetadata:')
print(json.dumps(json.loads((OUTPUT_DIR / 'run_metadata.json').read_text()), indent=2))
```

Listing 6: Inspect the GNNHAR and GNNHAR-IV loss table.

```
import pandas as pd

pd.read_csv(OUTPUT_DIR / 'tables/model_losses.csv').head(20)
```

Listing 7: Upload local outputs to Google Drive through the Drive API.

```
import json
import mimetypes

from google.colab import userdata
from google.oauth2 import service_account
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

service_account_json = userdata.get('GDRIVE_SERVICE_ACCOUNT_JSON')
folder_id = userdata.get('GDRIVE_OUTPUT_FOLDER_ID')
if not service_account_json:
    raise RuntimeError('Missing Colab Secret: GDRIVE_SERVICE_ACCOUNT_JSON')
```

```
if not folder_id:
    raise RuntimeError('Missing Colab Secret: GDRIVE_OUTPUT_FOLDER_ID')

info = json.loads(service_account_json)
creds = service_account.Credentials.from_service_account_info(
    info, scopes=['https://www.googleapis.com/auth/drive']
)
drive = build('drive', 'v3', credentials=creds)

def upload_file(path, parent_id):
    mime_type = mimetypes.guess_type(path.name)[0] or 'application/octet-stream'
    meta = {'name': path.name, 'parents': [parent_id]}
    media = MediaFileUpload(str(path), mimetype=mime_type, resumable=True)
    return drive.files().create(
        body=meta, media_body=media, fields='id, name'
    ).execute()

for path in sorted(OUTPUT_DIR.rglob('*')):
    if path.is_file():
        print('uploaded:', upload_file(path, folder_id))
```

References

AlphaQuery. (2026). Volatility and option statistics data variables. https://www.alphaquery.com/. Accessed June 2, 2026.

Bollerslev, T., Hood, B., Huss, J., and Pedersen, L. H. (2018). Risk everywhere: Modeling and managing volatility. *Review of Financial Studies*, 31(7), 2729–2773.

Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196.

Diebold, F. X., and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business and Economic Statistics*, 13(3), 253–263.

Engle, R. F. (2002). New frontiers for ARCH models. *Journal of Applied Econometrics*, 17(5), 425–446.

Friedman, J., Hastie, T., and Tibshirani, R. (2008). Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3), 432–441.

Hansen, P. R., Lunde, A., and Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497.

Zhang, C., Pu, X., Cucuringu, M., and Dong, X. (2025). Forecasting realized volatility with spillover effects: Perspectives from graph neural networks. *International Journal of Forecasting*, 41, 377–397.